

Selective Deliberation for Language Agents: Matching Fixed Test-Time Compute at Half the Cost

Anonymous ACL submission

Abstract

Language agents can spend additional test-time compute on reflection, search, debate, and verification, but it is unclear when this spending pays off. We study the question on a database tool-use benchmark of 300 tasks, holding a single hosted instruction-tuned language model constant across five inference-time conditions: a bare ReAct agent, three fixed-budget deliberation strategies (reflexion, tree search, and multi-agent debate), and a training-free adaptive controller that starts from a low-cost ReAct trajectory, estimates uncertainty from trajectory and verifier signals, selectively escalates, and stops when a verifier judges the answer sufficient. Under the benchmark’s official binary-success scorer, none of the accuracy differences among the five conditions are statistically distinguishable (paired McNemar exact tests, all $p \geq 0.18$): fixed deliberation does not raise task accuracy over the bare agent, and the only large, real differences are in compute cost. The adaptive controller reaches 33.3% success, within noise of the best observed condition, while spending about half the tokens and model calls of fixed search and debate. Our contribution is cost-efficiency at equal accuracy, not an accuracy gain; we report token, model-call, and wallclock accounting rather than dollar cost, and we characterize the verifier stop behavior and the dominant wrong-answer failure mode.

1 Introduction

Language agents now routinely couple a base tool-use policy with extra inference-time machinery: verbal self-reflection between attempts (Shinn et al., 2023), branching search over intermediate thoughts (Yao et al., 2023), and self-verification loops that re-check candidate answers (Zhao et al., 2026). A paral-

lel line of work frames this machinery as *test-time scaling*: spend more computation at inference and expect higher task success (Zhu et al., 2025). The appeal is obvious, because agent benchmarks reward the final answer and not the token bill, so any mechanism that occasionally rescues a failing trajectory looks worthwhile in aggregate.

Two observations complicate that picture. First, the base mechanism these methods build on is itself brittle: the acting-and-reasoning loop can be sensitive to prompt phrasing and can degrade when the environment deviates from the demonstrated pattern (Verma et al., 2024). Second, the components that add compute do not add it uniformly usefully. Tree search helps most when a model can *generate* good candidates but not necessarily when it must *discriminate* among them (Chen et al., 2024), and reflection-style memory can be made far cheaper without losing its benefit (Wu et al., 2025). Test-time compute is therefore not a free monotone knob; it is a resource whose marginal value depends on the task and on where in the trajectory it is spent (Dao et al., 2026).

This exposes a measurement gap. Agent evaluation surveys emphasize task success rates and coverage of capabilities (Mohammadi et al., 2025), while test-time-scaling work emphasizes how much inference-time computation is applied. Neither, on its own, answers the operational question a practitioner faces: given a fixed backend and a real benchmark, does adding reflection, search, or debate actually buy accuracy, and if not, can a controller recover the same accuracy for less compute by deciding *when* to deliberate rather than deliberating everywhere? Answering this requires a like-for-like comparison in which the model, prompts, task order, and official scorer are

held constant, and in which cost is measured, not assumed.

We run exactly that comparison on a database tool-use benchmark: an agent interacts with a live SQL database and returns an answer or commits a modification, scored by the benchmark’s official result comparator over 300 tasks. A single hosted instruction-tuned model is held fixed across five conditions: a bare ReAct agent; fixed reflexion; fixed tree search; fixed multi-agent debate; and *Adaptive (verifier-stop)*, a training-free controller that begins from one cheap ReAct trajectory, estimates uncertainty from trajectory and verifier signals, escalates only when the verifier is not satisfied, and stops as soon as it is. The insight we test is that the treatment is not “more thinking” but *routing and stopping*: easy tasks should terminate after one cheap pass, and only uncertain trajectories should pay for escalation.

The honest headline is a null result on accuracy paired with a real result on cost. Fixed reflexion, search, and debate do not produce distinguishable accuracy differences over the bare agent (paired McNemar exact tests, all $p \geq 0.18$), so the observed accuracy spread from 30.7% to 33.7% is within sampling noise. What differs sharply is compute: fixed search and debate cost 4346 and 4649 tokens per task at 4.0 model calls each, against 1124 tokens for the bare agent. The adaptive controller preserves the top of the observed accuracy range at 33.3% using 2144 tokens and 2.0 model calls, about half the tokens and calls of fixed search and debate. We frame this as cost-efficiency at equal accuracy, and we are explicit that it is not an accuracy improvement.

Contributions.

1. A controlled, single-backend comparison on a database tool-use benchmark (300 tasks, official binary-success scorer) of a bare ReAct agent against three fixed test-time deliberation strategies and a training-free adaptive verifier-stop controller, with per-task token, model-call, and wallclock accounting and paired significance testing.
2. The empirical finding that, on this benchmark, fixed reflexion, search, and de-

bate do not yield distinguishable accuracy gains over the bare agent while multiplying compute two- to four-fold, so the meaningful axis of variation is cost, not accuracy.

3. An adaptive controller that matches the best observed accuracy at about half the token and model-call cost of fixed search and debate, together with an analysis of its verifier stop behavior (295 accept, 5 reject over 300 tasks) and the dominant wrong-answer failure mode shared by every condition.

Scope. Our claims are confined to one benchmark family and one backend. We do not claim an accuracy improvement, we do not use monetary cost (route pricing was unavailable, so we report compute), and we treat two additional benchmark families as planned rather than completed evaluation (Section 7). We never use the word “significant” for the accuracy differences reported here, because the tests do not support it.

2 Related Work

Acting-and-reasoning agents and their fragility. The base policy in our study is a ReAct-style agent that interleaves reasoning steps with tool calls. This paradigm underlies most tool-use agents, but its foundations are less stable than aggregate benchmark numbers suggest: small changes to prompting and to the gap between demonstrated and encountered situations can substantially move behavior (Verma et al., 2024). This fragility is precisely why extra test-time compute is attractive as a corrective, and also why a cheap first pass plus a decision rule, rather than a uniformly heavier pipeline, is a reasonable design point. The gap we address is that the fragility argument is usually made in the abstract; we quantify, on a real scorer, how much a corrective layer actually recovers.

Reflection, search, and debate as fixed deliberation. Reflexion adds verbal feedback between attempts, letting an agent revise after an unsuccessful trajectory (Shinn et al., 2023); Tree-of-Thoughts replaces a single chain with a search over intermediate states (Yao et al., 2023). Both are typically

applied as fixed procedures. Subsequent analysis shows their value is conditional: tree search helps when the bottleneck is generating good candidates rather than discriminating among them (Chen et al., 2024), and reflective memory can be distilled into reusable rules that cut its resource cost (Wu et al., 2025). Policy-guided variants make the same point by bounding the number of model queries a search is allowed to spend (Pendurkar and Sharon, 2026). These works argue, component by component, that fixed deliberation is often wasteful. Our contribution is a head-to-head, cost-instrumented comparison of all three fixed strategies against a bare agent on one benchmark, which lets us ask whether the wasted compute buys any accuracy at all here; it does not.

Test-time compute allocation. A growing literature treats inference-time computation as a budget to be allocated rather than a switch to be flipped. Work on scaling test-time compute for agents studies how additional inference passes translate into task success (Zhu et al., 2025), and allocation studies ask specifically *when* and *where* extra compute should be spent in a planner (Dao et al., 2026). Black-box search over agent rollouts has been used to approximate the effect of learning without updating weights (Hwang et al., 2026). Our controller sits in this family but is deliberately training-free and minimal: it allocates by escalating only when a verifier is unsatisfied, and its empirical lesson here is that the correct allocation on many tasks is *zero* extra deliberation.

Verification and uncertainty as the stopping signal. Self-verification can itself be scaled: fine-grained verification improves agentic search by re-checking intermediate steps (Zhao et al., 2026), and speculative-execution schemes gate an untrusted agent behind a certifying check before its actions are trusted (Zhou et al., 2026). Deciding when to stop or escalate also requires a usable notion of uncertainty; decomposing agent uncertainty has been proposed as a trigger for clarification-seeking (Matsnev, 2026). Adaptive-rubric evaluation similarly argues that the checking standard should vary by task (Ding, 2026). We use a verifier both as the escalation trigger

and as the stopping criterion, and we report the verifier’s realized accept/reject behavior so that its role is auditable rather than assumed.

Agent benchmarks and tool-use evaluation. Our evaluation family comes from AgentBench, which casts LLMs as agents across interactive environments including database interaction (Liu et al., 2023). Tool-augmented evaluation more broadly includes API-centric benchmarks (Li et al., 2023), MCP-mediated real-world tool suites (Guo et al., 2025), general-assistant tasks (Mialon et al., 2023), and software-repair benchmarks whose live variants stress long-horizon coding (Zhang et al., 2025). A recent survey catalogs how these benchmarks measure success and where they fall short (Mohammadi et al., 2025). The consistent gap across this literature is that cost is reported inconsistently or not at all, so a method that trades accuracy for compute cannot be compared fairly. We close that gap for one family by holding the backend fixed and reporting tokens, model calls, and wallclock alongside official success.

3 Method

Setup and notation. For a task x the agent interacts with a database environment and produces a committed answer (a query result or a modification). A base ReAct policy π_0 produces a trajectory $\tau_0 = \pi_0(x)$ with committed answer y_0 . A verifier V maps any trajectory τ to a scalar confidence $s(\tau) \in [0, 1]$ and a binary accept decision

$$A(\tau) = \mathbf{1}[s(\tau) \geq \theta],$$

where θ is a predeclared acceptance threshold. An uncertainty functional $u(\tau)$ aggregates trajectory-level signals into a scalar escalation trigger,

$$u(\tau) = g(d(\tau), 1 - s(\tau), e(\tau), r(\tau), i(\tau)),$$

where $d(\tau)$ is answer/tool disagreement, $s(\tau)$ the verifier confidence, $e(\tau)$ a tool-error indicator, $r(\tau)$ a repeated-failed-action count, and $i(\tau)$ a self-reported incompleteness flag; g is a fixed monotone combiner with no learned parameters. An ordered escalation set $E = (\text{REFLEXION}, \text{SEARCH}, \text{DEBATE})$ supplies candidate-generating branches, and a per-task

budget cap B bounds the number of model calls.

Decision rule. The controller, *Adaptive (verifier-stop)*, is training-free and proceeds as a stop-or-escalate loop:

1. Generate the base trajectory τ_0 with π_0 and score it with the verifier to obtain $s(\tau_0)$ and $A(\tau_0)$.
2. **Stop test.** If $A(\tau_0) = 1$, commit y_0 and terminate. This is the common case for easy tasks: one cheap trajectory plus one verifier check.
3. **Escalation.** Otherwise, compute $u(\tau_0)$ and escalate to the next branch $b \in E$ (reflexion first, then search, then debate), generating a new candidate trajectory τ_b conditioned on the accumulated evidence.
4. **Re-verify and stop.** Score τ_b ; if $A(\tau_b) = 1$, commit and terminate. Otherwise retain the best-so-far candidate $\arg \max_{\tau} s(\tau)$ and continue.
5. **Budget stop.** Terminate when the accumulated model-call count $c(\tau)$ reaches B , committing the highest-verifier-score candidate observed.

The committed answer is thus $\hat{y} = y_{\tau^*}$ with $\tau^* = \arg \max_{\tau} s(\tau)$ over all trajectories explored for x . Two properties matter for interpretation. First, the controller never spends more than the fixed baselines, because escalation is entered only when the verifier rejects and is capped at B ; on tasks it accepts immediately it spends the base trajectory plus one verifier pass. Second, the controller can only commit an escalated answer when a later candidate is both produced and preferred by the verifier; if escalation yields nothing the verifier prefers, the committed answer equals y_0 .

Fixed baselines. The four comparison conditions share π_0 , the prompts, the task order, and the backend, and differ only in test-time compute:

- *Bare ReAct*: a single ReAct trajectory with no reflection, search, or debate; the reference cost floor.
- *Fixed reflexion*: Reflexion-style verbal feedback applied on every task with a fixed second attempt (Shinn et al., 2023).

- *Fixed search*: Tree-of-Thoughts-style search with fixed breadth and depth on every task (Yao et al., 2023).

- *Fixed debate*: multi-agent debate with a fixed number of agents and rounds on every task.

Each fixed baseline is matched to the adaptive controller’s maximum escalation so that any difference is attributable to *when* compute is spent, not to how much is available. Because the controller reuses the same branches as the fixed baselines, the study is a clean test of the routing-and-stopping hypothesis: if selective escalation matched fixed escalation on accuracy while spending less, the saving would be real; if fixed escalation did not beat the bare agent, the escalation itself would be the questionable ingredient. We report both outcomes.

Uncertainty signals and the verifier.

The escalation trigger $u(\tau)$ and the verifier $A(\tau)$ are the two decision points that make the controller adaptive, and both are training-free. The trigger combines five signals that are cheap to read off a completed trajectory: answer/tool disagreement $d(\tau)$ (whether repeated tool queries returned inconsistent results); the verifier’s own residual $1 - s(\tau)$; a tool-error indicator $e(\tau)$ (a failed or malformed SQL call); a repeated-failed-action count $r(\tau)$ (the same action retried without progress); and a self-reported incompleteness flag $i(\tau)$ (the agent signaling it could not finish). Because g is a fixed monotone combiner, the controller has no free parameters to tune on the benchmark; only θ is set once, in advance. The verifier is realized by the same backend prompted to judge whether the committed answer is supported by the observed database interaction, returning $s(\tau)$ and, via θ , the accept decision $A(\tau)$. Using the verifier for both the trigger and the stop is deliberate: it means a single additional model call both decides whether the cheap trajectory is trustworthy and, if so, ends the task, which is why the accepted-task cost is one base trajectory plus one verifier pass rather than a full escalation. This design also makes the verifier the single point whose realized behavior we can and do audit (Section 6).

Field	Value
Family	AgentBench database tool use
Slice	dbbench-std
Tasks	300
License	Apache-2.0 (repository)
Official scorer	DBResultProcessor comparison
Backend	one hosted instruction-tuned LLM, fixed
Conditions	5 (bare, reflexion, search, debate, adaptive)
Rows	1500 (300 × 5)

Table 1: Benchmark provenance for the completed evaluation. The public slice, license, and official scorer are taken from the source repository; the five conditions run on the identical 300 tasks in the same order.

Cost accounting. For each task and condition we record prompt, completion, and total tokens, the model-call count $m(x)$, and wallclock seconds $w(x)$, aggregated to per-task means. Monetary cost is intentionally omitted: the hosted route did not expose per-response pricing, so we report compute (tokens, calls, wallclock) and forbid dollar-denominated claims. This keeps the comparison honest about what was actually measured.

4 Experimental Setup

Benchmark and scorer. We evaluate on the database tool-use family of AgentBench (Liu et al., 2023), standard split, comprising 300 tasks loaded from the official repository. Each task’s success is decided by the benchmark’s own result comparator (the official DBResultProcessor answer/modification comparison), not by any bespoke metric of ours, so scores are on the benchmark’s terms. Provenance is summarized in Table 1.

Backend. All conditions use a single hosted instruction-tuned language model accessed through a chat/responses API, held fixed across the five conditions with identical decoding parameters and a fixed seed. We do not vary the model, so every accuracy or cost difference is attributable to the inference-time procedure rather than to model choice. The same backend serves the base policy, the escalation branches, and the verifier.

Conditions and matching. The five conditions are Bare ReAct, Fixed reflexion, Fixed search, Fixed debate, and Adaptive (verifier-

stop), as defined in Section 3. All share prompts, task order, and backend; the fixed baselines apply their deliberation to every task, while the adaptive controller applies it selectively. The adaptive escalation cap is matched to the fixed baselines’ budget so no condition is allowed strictly more compute headroom than another.

Metrics. The primary metric is binary task success under the official scorer, reported as a percentage over the 300 tasks. Cost is reported as mean total tokens per task, mean model calls per task, and mean wallclock seconds per task. We additionally report the verifier’s realized accept/reject counts and the per-row failure labels emitted alongside each scored trajectory.

Compute budget and stopping. The full run was executed under a predeclared model-call cap of 4000 hosted calls for the whole five-condition matrix, sized to just exceed the planned call count; the run wrote raw per-task rows for all 1500 task-condition pairs. Each condition ran the same 300 tasks under one seed; we therefore compare conditions with paired, per-task tests rather than unpaired averages.

Significance testing. Because the conditions are paired on tasks, we test accuracy differences with McNemar’s exact test on the per-task success indicators, reporting the number of discordant pairs and the two-sided exact p -value for each comparison of interest. We do not apply parametric assumptions to the small discordant counts.

5 Results

Table 2 reports the five conditions on the identical 300 tasks. The accuracy column spans a narrow band, from 30.7% to 33.7%, while the cost columns span a wide range: tokens per task increase more than four-fold from the bare agent to fixed debate, and model calls per task range from 1.0 to 4.0.

Accuracy is flat. No fixed deliberation strategy beats the bare agent by a distinguishable margin. The paired McNemar exact tests give: fixed search vs. bare ReAct, $p = 0.180$ (9 discordant pairs); fixed reflexion vs. bare Re-

Condition	Succ. (%)	Tok. /task	Calls /task	Secs /task
Bare ReAct	32.0	1124	1.0	5.7
Fixed reflexion	30.7	2343	2.0	12.2
Fixed search	33.7	4346	4.0	18.4
Fixed debate	31.7	4649	4.0	20.4
Adaptive (verifier-stop)	33.3	2144	2.0	9.1

Table 2: DBBench (dbbench-std, 300 tasks), one backend held constant. The takeaway is that accuracy is flat across conditions while cost is not: fixed search and debate quadruple tokens and model calls without a distinguishable accuracy gain, whereas the adaptive controller holds the top of the accuracy band at about half their cost. Bold marks the highest observed success (fixed search, 33.7%); the adaptive controller at 33.3% is statistically indistinguishable from it.

Act, $p = 0.481$ (18 discordant); and fixed debate vs. bare ReAct, $p = 1.000$ (7 discordant). Fixed reflexion is in fact numerically below the bare agent (30.7% vs. 32.0%). On this benchmark, adding fixed reflection, search, or debate did not improve accuracy.

Cost is not flat. The same table shows the real effect of these strategies is on compute. Fixed search and fixed debate cost 4346 and 4649 tokens per task at 4.0 model calls and 18.4 and 20.4 seconds; the bare agent costs 1124 tokens, 1.0 call, and 5.7 seconds. The extra compute is spent, but not converted into distinguishable accuracy.

The adaptive controller. Adaptive (verifier-stop) reaches 33.3% success, statistically indistinguishable from the best observed condition (fixed search at 33.7%; McNemar $p = 1.000$, 5 discordant pairs), and indistinguishable from the bare agent ($p = 0.344$, 10 discordant). It does so at 2144 tokens, 2.0 model calls, and 9.1 seconds per task, roughly half the tokens and model calls of fixed search (4346 tokens, 4.0 calls) and fixed debate (4649 tokens, 4.0 calls). The finding is cost-efficiency at equal accuracy, not an accuracy gain.

6 Analysis

Cost-quality reading. Placed on a cost-accuracy plane, four of the five conditions form a nearly horizontal line: accuracy stays inside a 3.0-point band (30.7% to 33.7%) while

per-task tokens move from 1124 to 4649. Because the vertical spread is not distinguishable from noise (next paragraph), the only defensible way to prefer one condition over another is horizontal, i.e. by cost. On that axis the ordering is clear: the bare agent is cheapest, the adaptive controller is next at 2144 tokens, and fixed search and debate are roughly twice as expensive at 4346 and 4649 tokens for no accuracy return. The adaptive controller thus dominates fixed search and fixed debate: at least as accurate (indeed within noise of the best) and about half the cost.

Paired significance statement. We state the significance result explicitly and without hedging in either direction. Across all pairwise comparisons of interest the paired McNemar exact test never rejects equality of accuracy: adaptive vs. fixed search $p = 1.000$ (5 discordant); adaptive vs. bare ReAct $p = 0.344$ (10 discordant); fixed search vs. bare ReAct $p = 0.180$ (9 discordant); fixed reflexion vs. bare ReAct $p = 0.481$ (18 discordant); fixed debate vs. bare ReAct $p = 1.000$ (7 discordant). The smallest p -value is 0.180. None of the accuracy differences in Table 2 are statistically distinguishable, and we do not describe any of them as significant. The efficiency claim rests entirely on the cost columns, where the differences are large and directly measured.

What the controller actually did. The verifier’s realized behavior explains both the low cost and the equal accuracy. Over the 300 adaptive tasks the verifier accepted 295 candidates and rejected 5; every task terminated on a verifier decision (stop reason “verifier accept” for the 295 and “verifier reject” for the 5), and the committed candidate was unchanged from the initial ReAct answer on all 300 tasks. In other words, on this benchmark the controller behaved almost entirely as a bare trajectory followed by one verifier check: it rarely needed to escalate, and when it did (5 tasks) the escalation did not overturn the committed answer. This is why its cost sits just above the bare agent (2.0 vs. 1.0 calls, the extra call being the verifier) rather than near the fixed baselines, and why its accuracy tracks the bare agent. The 1.3-point gap between the adaptive controller (33.3%) and the standalone bare condition (32.0%) is

not produced by escalation, since the committed answer never changed; it reflects independent sampling of the base trajectory in the two runs, consistent with the paired test that finds them indistinguishable ($p = 0.344$). The practical implication is that a verifier used purely as a stopping gate is cheap and safe here, but on this task distribution it has little to escalate to: the deliberation branches it could invoke do not, on average, help.

Headroom for routing is small. A natural question is whether *any* allocation policy could have won here, since routing can only help on tasks whose outcome depends on the condition. We therefore partition the 300 tasks by cross-condition agreement. Only 82 tasks are solved by all five conditions, and 193 tasks are solved by none; just 107 tasks are solved by at least one condition. Consequently only $107 - 82 = 25$ tasks are condition-sensitive at all, and an oracle that picked the best condition per task would reach only $107/300 = 35.7\%$, barely above the best single condition (33.7%). This is the structural reason no deliberation strategy separates: the large hard set (193 tasks) is unmoved by reflection, search, or debate, and the small easy set (82 tasks) needs none, so extra compute is spent almost entirely on tasks whose outcome it cannot change. Selective stopping is attractive precisely in this regime, because the correct decision on the vast majority of tasks is to not deliberate.

Why fixed deliberation did not help: the failure mode. The dominant failure across all conditions is a wrong final answer, not a crashed or truncated trajectory. Of the 1500 scored rows, 1016 carry the wrong-answer label (204, 208, 199, 205, and 200 rows for the bare, reflexion, search, debate, and adaptive conditions respectively), and it is the single most frequent failure label in every condition. This matters for interpreting the null result. Reflection, search, and debate primarily help when the agent can recognize that it is wrong and then find a better path; they help far less when the agent confidently commits an incorrect database answer that neither it nor the verifier flags. The prevalence of confident wrong answers is consistent with prior observations that models are stronger at generat-

ing candidates than at discriminating among them (Chen et al., 2024), and it bounds what a verifier-gated controller can recover: a stopping gate cannot rescue an error that the verifier accepts. This also delimits our efficiency claim honestly, since the controller’s savings come from not paying for deliberation that would not have changed the answer, not from fixing more answers.

Takeaways. Three points follow. (i) On this benchmark, “more test-time compute” is not itself a treatment for accuracy; fixed reflexion, search, and debate buy cost, not correctness. (ii) A training-free verifier-stop controller recovers the best observed accuracy at about half the cost of the heavy fixed strategies, so when deliberation is available but often useless, deciding *when* to deliberate is the operative lever. (iii) The residual error is dominated by confident wrong answers that a stopping verifier does not catch, which points future work toward better verification and error detection rather than toward more deliberation breadth.

7 Limitations

Single family, single backend. Our completed evidence is one benchmark family (database tool use, 300 tasks) under one hosted backend. We do not claim the null-accuracy result generalizes to reasoning-heavy or long-horizon settings where deliberation is known to help more; the controller’s advantage here is specifically cost-efficiency at equal accuracy on this distribution, and a different task mix could change both the accuracy verdict and the escalation rate.

Planned families not yet completed. The study was designed for three independent public families. Only the database family produced completed, officially scored rows. A tool-agent-user interaction family (tau2-bench) was prepared but not run for paper-facing rows: a conservative forecast of the full five-condition matrix required about 8062 hosted model calls, exceeding the authorized 3614-call cap, so it remains ongoing rather than reported. A software-repair family (SWE-bench Verified) was prepared but did not reach its official containerized scorer in

this environment, and we deliberately report no numbers for it. We present both families only as planned or in-progress evaluation and invent no rows for them.

Verifier ceiling and confident errors. Because the dominant failure is a confident wrong answer that the verifier accepts (1016 of 1500 rows carry the wrong-answer label), the controller cannot correct errors it does not detect. Our verifier is training-free and imperfect; a stronger or calibrated verifier might both escalate more often and stop more safely, which we did not test here.

Cost is compute, not money. The hosted route did not expose per-response pricing, so we report tokens, model calls, and wallclock and make no monetary claims. Token-to-dollar conversions depend on provider pricing we could not observe, and wallclock is sensitive to serving load; the token and call counts are the most stable cost signals.

8 Conclusion

We asked whether extra test-time compute pays off for a language agent on a database tool-use benchmark, holding the backend fixed and measuring cost. It did not pay off in accuracy: fixed reflexion, search, and debate were statistically indistinguishable from a bare ReAct agent (all paired $p \geq 0.18$), and differed only in compute, up to four-fold in tokens and model calls. A training-free adaptive controller that starts cheap, escalates only on verifier rejection, and stops on verifier acceptance preserved the best observed accuracy (33.3%) at about half the tokens and model calls of the heavy fixed strategies. The contribution is cost-efficiency at equal accuracy, not an accuracy improvement, and the residual errors are dominated by confident wrong answers a stopping verifier does not catch. On this evidence, the useful question for agent test-time compute is not how much to spend but when to spend nothing.

Ethics and Reproducibility

This work uses a public benchmark under its stated repository license and its official scorer; no human subjects or private data are involved. We report every result number

from the same raw per-task log covering all 1500 task-condition rows, with per-task token, model-call, and wallclock accounting, a fixed seed, and paired McNemar exact tests for every accuracy comparison. We deliberately report compute rather than monetary cost because the hosted route did not expose pricing, and we present the two unfinished benchmark families as planned evaluation without fabricating any numbers for them.

References

- Qiqi Chen, Xinpeng Wang, Philipp Mondorf, Michael A. Hedderich, and Barbara Plank. 2024. [Understanding When Tree of Thoughts Succeeds: Larger Models Excel in Generation, Not Discrimination](#).
- Jadelynn Dao, Milan Ganai, Yasmina Abukhadra, Ajay Sridhar, Mozghan Nasr Azadani, Katie Luo, Clark Barrett, Jiajun Wu, Chelsea Finn, and Marco Pavone. 2026. [DIRECT: When and Where Should You Allocate Test-Time Compute in Embodied Planners?](#)
- Liang Ding. 2026. [AdaRubric: Task-Adaptive Rubrics for Reliable LLM Agent Evaluation and Reward Learning](#).
- Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhendong Mao. 2025. [MCP-AgentBench: Evaluating Real-World Language Agent Performance with MCP-Mediated Tools](#).
- Dae Yon Hwang, Raunaq Suri, Valentin Villecroze, Anthony L. Caterini, Jesse C. Cresswell, Noël Vouitsis, and Brendan Leigh Ross. 2026. [Agentic Monte Carlo: Simulating Reinforcement Learning for Black-Box Agents](#).
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. [API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs](#).
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023. [AgentBench: Evaluating LLMs as Agents](#).
- Gregory Matsnev. 2026. [Uncertainty Decomposition for Clarification Seeking in LLM Agents](#).
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. [GAIA: a benchmark for General AI Assistants](#).

- Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. [Evaluation and Benchmarking of LLM Agents: A Survey](#).
- Sumedh Pendurkar and Guni Sharon. 2026. [Policy-Guided Search on Tree-of-Thoughts for Efficient Problem Solving with Bounded Language Model Queries](#).
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language Agents with Verbal Reinforcement Learning](#).
- Mudit Verma, Siddhant Bhambri, and Subbarao Kambhampati. 2024. [On the Brittle Foundations of ReAct Prompting for Agentic Large Language Models](#).
- Chunlong Wu, Ye Luo, Zhibo Qu, and Min Wang. 2025. [Meta-Policy Reflexion: Reusable Reflective Memory and Rule Admissibility for Resource-Efficient LLM Agent](#).
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. [Tree of Thoughts: Deliberate Problem Solving with Large Language Models](#).
- Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. [SWE-bench Goes Live!](#)
- James Xu Zhao, Hui Chen, Bryan Hooi, and See-Kiong Ng. 2026. [FineVerify: Scaling Test-Time Compute with Fine-Grained Self-Verification for Agentic Search](#).
- Chenyu Zhou, Qiliang Jiang, Shuning Wu, and Xu Zhou. 2026. [Certified Speculative Execution for Untrusted AI Agents](#).
- King Zhu, Hanhao Li, Siwei Wu, Tianshun Xing, Dehua Ma, Xiangru Tang, Minghao Liu, Jian Yang, Jiaheng Liu, Yuchen Eleanor Jiang, Changwang Zhang, Chenghua Lin, Jun Wang, Ge Zhang, and Wangchunshu Zhou. 2025. [Scaling Test-time Compute for LLM Agents](#).